



The Coq Proof Assistant

Library Coq.Init.Peano

The type `nat` of Peano natural numbers (built from `0` and `S`) is defined in `Datatypes.v`

This module defines the following operations on natural numbers :

- predecessor `pred`
 - addition `plus`
 - multiplication `mult`
 - less or equal order `le`
 - less `lt`
 - greater or equal `ge`
 - greater `gt`

It states various lemmas and theorems about natural numbers, including Peano's axioms of arithmetic (in Coq, these are provable). Case analysis on `nat` and induction on `nat * nat` are provided too

```
Require Import Notations.
Require Import Ltac.
Require Import Datatypes.
Require Import Logic.
Require Coq.Init.Nat.
```

```
Open Scope nat_scope.
```

```
Definition eq_S := f_equal S.
Definition f_equal_nat := f_equal (A:=nat).
```

```
Hint Resolve f_equal_nat: core.
```

The predecessor function

```
Notation pred := Nat.pred (only parsing).
```

```
Definition f_equal_pred := f_equal pred.
```

```
Theorem pred_Sn : forall n:nat, n = pred (S n).
```

Injectivity of successor

```
Definition eq_add_S n m (H: S n = S m): n = m :=
  f_equal pred H.
Hint Immediate eq_add_S: core.
```

Navigation

- Standard Library
 - [Table of contents](#)
 - [Index](#)

```
Theorem not_eq_S : forall n m:nat, n <> m -> S n <> S m.
```

```
Hint Resolve not_eq_S: core.
```

```
Definition IsSucc (n:nat) : Prop :=  
  match n with  
  | 0 => False  
  | S p => True  
  end.
```

Zero is not the successor of a number

```
Theorem 0_S : forall n:nat, 0 <> S n.
```

```
Hint Resolve 0_S: core.
```

```
Theorem n_Sn : forall n:nat, n <> S n.
```

```
Hint Resolve n_Sn: core.
```

Addition

```
Notation plus := Nat.add (only parsing).
```

```
Infix "+" := Nat.add : nat_scope.
```

```
Definition f_equal2_plus := f_equal2 plus.
```

```
Definition f_equal2_nat := f_equal2 (A1:=nat)  
(A2:=nat).
```

```
Hint Resolve f_equal2_nat: core.
```

```
Lemma plus_n_0 : forall n:nat, n = n + 0.
```

```
Hint Resolve plus_n_0 eq_refl: core.
```

```
Lemma plus_0_n : forall n:nat, 0 + n = n.
```

```
Lemma plus_n_Sm : forall n m:nat, S (n + m) = n + S m.
```

```
Hint Resolve plus_n_Sm: core.
```

```
Lemma plus_Sn_m : forall n m:nat, S n + m = S (n + m).
```

Standard associated names

```
Notation plus_0_r_reverse := plus_n_0 (only parsing).
```

```
Notation plus_succ_r_reverse := plus_n_Sm (only  
parsing).
```

Multiplication

```
Notation mult := Nat.mul (only parsing).
```

```
Infix "*" := Nat.mul : nat_scope.
```

```
Definition f_equal2_mult := f_equal2 mult.
```

```
Hint Resolve f_equal2_mult: core.
```

Navigation

- [Standard Library](#)
 - [Table of contents](#)
 - [Index](#)

```
Lemma mult_n_0 : forall n:nat, 0 = n * 0.
```

```
Hint Resolve mult_n_0: core.
```

```
Lemma mult_n_Sm : forall n m:nat, n * m + n = n * S m.
```

```
Hint Resolve mult_n_Sm: core.
```

Standard associated names

```
Notation mult_0_r_reverse := mult_n_0 (only parsing).
```

```
Notation mult_succ_r_reverse := mult_n_Sm (only  
parsing).
```

Truncated subtraction: $m - n$ is 0 if $n \geq m$

```
Notation minus := Nat.sub (only parsing).
```

```
Infix "-" := Nat.sub : nat_scope.
```

Definition of the usual orders, the basic properties of `le` and `lt`
can be found in files `Le` and `Lt`

```
Inductive le (n:nat) : nat -> Prop :=
```

```
| le_n : n <= n
```

```
| le_S : forall m:nat, n <= m -> n <= S m
```

```
where "n <= m" := (le n m) : nat_scope.
```

```
Hint Constructors le: core.
```

```
Definition lt (n m:nat) := S n <= m.
```

```
Hint Unfold lt: core.
```

```
Infix "<" := lt : nat_scope.
```

```
Definition ge (n m:nat) := m <= n.
```

```
Hint Unfold ge: core.
```

```
Infix ">=" := ge : nat_scope.
```

```
Definition gt (n m:nat) := m < n.
```

```
Hint Unfold gt: core.
```

```
Infix ">" := gt : nat_scope.
```

```
Notation "x <= y <= z" := (x <= y /\ y <= z) :  
nat_scope.
```

```
Notation "x <= y < z" := (x <= y /\ y < z) :  
nat_scope.
```

```
Notation "x < y < z" := (x < y /\ y < z) : nat_scope.
```

```
Notation "x < y <= z" := (x < y /\ y <= z) :  
nat_scope.
```

Navigation

- Standard Library
 - Table of contents
 - Index

Theorem le_pred : forall n m, n <= m -> pred n <= pred m.

Theorem le_S_n : forall n m, S n <= S m -> n <= m.

Theorem le_0_n : forall n, 0 <= n.

Theorem le_n_S : forall n m, n <= m -> S n <= S m.

Case analysis

Theorem nat_case :
forall (n:nat) (P:nat -> Prop), P 0 -> (forall m:nat,
P (S m)) -> P n.

Principle of double induction

Theorem nat_double_ind :
forall R:nat -> nat -> Prop,
 (forall n:nat, R 0 n) ->
 (forall n:nat, R (S n) 0) ->
 (forall n m:nat, R n m -> R (S n) (S m)) -> forall
n m:nat, R n m.

Maximum and minimum : definitions and specifications

Notation max := Nat.max (only parsing).

Notation min := Nat.min (only parsing).

Lemma max_l n m : m <= n -> Nat.max n m = n.

Lemma max_r n m : n <= m -> Nat.max n m = m.

Lemma min_l n m : n <= m -> Nat.min n m = n.

Lemma min_r n m : m <= n -> Nat.min n m = m.

Lemma nat_rect_succ_r {A} (f: A -> A) (x:A) n :
 nat_rect (fun _ => A) x (fun _ => f) (S n) =
nat_rect (fun _ => A) (f x) (fun _ => f) n.

Theorem nat_rect_plus :
forall (n m:nat) {A} (f:A -> A) (x:A),
 nat_rect (fun _ => A) x (fun _ => f) (n + m) =
 nat_rect (fun _ => A) (nat_rect (fun _ => A) x
(fun _ => f) m) (fun _ => f) n.

Navigation

- Standard Library
 - Table of contents
 - Index